

[Home](#) > Efficient...

Efficient LLM Serving

Overview

With rapid development of generative AI, new techniques and algorithms for performance optimization and better resource utilization are introduced to make best use of the hardware and provide best generation performance. OpenVINO implements those state of the art methods in its [GenAI Library](#) like:

- Continuous Batching
- Paged Attention
- Dynamic Split Fuse
- *and more...*

It is now integrated into OpenVINO Model Server providing efficient way to run generative workloads.

Check out the [quickstart guide](#) for a simple example that shows how to use this feature.

Servable Types

Starting with 2025.1, we can highlight four servable types. Such distinction is made based on the input type and underlying GenAI pipeline. The servable types are:

- Language Model Continuous Batching,
- Language Model Stateful,
- Visual Language Model Continuous Batching,
- Visual Language Model Stateful.

First part - Language Model / Visual Language Model - determines whether servable accepts only text or both text and images on the input. Second part - Continuous Batching / Stateful - determines what kind of GenAI pipeline is used as the engine. By default CPU and GPU devices work on Continuous Batching pipelines. NPU device works only with the Stateful servable type.

[Skip to main content](#)

User does not have to explicitly select servable type. It is inferred based on model directory contents and selected target device. Model directory contents determine if model can work only with text or visual input as well. As for target device, setting it to **NPU** will always pick Stateful servable, while any other device will result in deploying Continuous Batching servable.

Stateful servables ignore most of the configuration used by Continuous Batching, but this will be mentioned later. Some servable types have additional limitations mentioned in the limitations section at the end of this document.

Despite all the differences, all servable types share the same LLM calculator which imposes certain flow in every GenAI-based endpoint.

LLM Calculator

As you can see in the quickstart, big part of the configuration resides in **graph.pbtxt** file. That's because model server text generation servables are deployed as MediaPipe graphs with dedicated LLM calculator that works with latest [OpenVINO GenAI](#) library. The calculator is designed to run in cycles and return the chunks of responses to the client.

On the input it expects a `HttpPayload` struct passed by the Model Server frontend:

```
struct HttpPayload {
    std::string uri;
    std::vector<std::pair<std::string, std::string>> headers;
    std::string body; // always
    std::shared_ptr<rapidjson::Document> parsedJson; // pre-parsed body
    = null
    std::shared_ptr<ClientConnection> client;
};
```

The input json content should be compatible with the [chat completions](#) or [completions](#) API.

The input also includes a side packet with a reference to **LLM_NODE_RESOURCES** which is a shared object representing an LLM engine. It loads the model, runs the generation cycles and reports the generated results to the LLM calculator via a generation handler.

Every node based on LLM Calculator MUST have exactly that specification of this side packet:

```
input_side_packet: "LLM_NODE_RESOURCES:llm"
```

If it's modified, model server will fail to provide graph with the model

[Skip to main content](#)

On the output the calculator creates an `std::string` with the json content, which is returned to the client as one response or in chunks with streaming.

Let's have a look at the simplest graph:

```
input_stream: "HTTP_REQUEST_PAYLOAD:input"
output_stream: "HTTP_RESPONSE_PAYLOAD:output"

node: {
  name: "LLMExecutor"
  calculator: "HttpLLMCalculator"
  input_stream: "LOOPBACK:loopback"
  input_stream: "HTTP_REQUEST_PAYLOAD:input"
  input_side_packet: "LLM_NODE_RESOURCES:llm"
  output_stream: "LOOPBACK:loopback"
  output_stream: "HTTP_RESPONSE_PAYLOAD:output"
  input_stream_info: {
    tag_index: 'LOOPBACK:0',
    back_edge: true
  }
  node_options: {
    [type.googleapis.com / mediapipe.LLMCalculatorOptions]: {
      models_path: "./"
    }
  }
  input_stream_handler {
    input_stream_handler: "SyncSetInputStreamHandler",
    options {
      [mediapipe.SyncSetInputStreamHandlerOptions.ext] {
        sync_set {
          tag_index: "LOOPBACK:0"
        }
      }
    }
  }
}
}
```

Above node configuration should be used as a template since user is not expected to change most of it's content. Actually only `node_options` requires user attention as it specifies LLM engine parameters. The rest of the configuration can remain unchanged.

The calculator supports the following `node_options` for tuning the pipeline configuration:

- `required string models_path` - location of the model directory (can be relative);
- `optional uint64 max_num_batched_tokens` - max number of tokens processed in a single iteration [default = 256];
- `optional uint64 cache_size` - memory size in GB for storing KV cache. If not set or set to 0, cache is allocated dynamically [default = 0];

[Skip to main content](#)

- `optional uint64 max_num_seqs` - max number of sequences actively processed by the engine [default = 256];
- `optional bool dynamic_split_fuse` - use Dynamic Split Fuse token scheduling [default = true];
- `optional string device` - device to load models to. Supported values: "CPU", "GPU" [default = "CPU"]
- `optional string plugin_config` - [OpenVINO device plugin configuration](#) and additional pipeline options. Should be provided in the same format for regular [models configuration](#) [default = "{}"]
- `optional uint32 best_of_limit` - max value of best_of parameter accepted by endpoint [default = 20];
- `optional uint32 max_tokens_limit` - max value of max_tokens parameter accepted by endpoint;
- `optional bool enable_prefix_caching` - enable caching of KV-blocks [default = false];
- `optional CacheEvictionConfig cache_eviction_config` - KV cache eviction configuration. Disabled if not specified.
- `optional string reasoning_parser` - name of the parser to use for reasoning content extraction from model output before creating a response;
- `optional string tool_parser` - name of the parser to use for tool calls extraction from model output before creating a response;
- `optional bool enable_tool_guided_generation` - enable enforcing tool schema during generation. Requires setting response parser. [default = false];
- `optional SparseAttentionConfig sparse_attention_config` - Sparse attention configuration. Disabled if not specified.

Caching settings

The value of `cache_size` might have performance and stability implications. It is used for storing LLM model KV cache data. Adjust it based on your environment capabilities, model size and expected level of concurrency. You can track the actual usage of the cache in the server logs. You can observe in the logs output like below:

```
[2024-07-30 14:28:02.536][624][llm_executor][info][llm_executor.hpp:65] All requests: 50; Scheduled requests: 25; Cache usage 23.9%;
```

The logs represent the cache usage for the current generation cycles. By default, the allocation is increased dynamically, which means more space is added when the context for all batched

[Skip to main content](#)

lead to 100% usage. When the cache is consumed, some of the running requests might be preempted to free cache for other requests to finish their generations (preemption will likely have negative impact on performance since preempted request cache will need to be recomputed when it gets processed again). When preemption is not possible i.e. `cache_size` is very small and there is a single, long running request that consumes it all, then the request gets terminated when no more cache can be assigned to it, even before reaching stopping criteria. The dynamic allocation reserves as much as required to prevent preemption. In such way it should be most efficient to get optimal performance. In some cases it could lead however to consuming all available RAM.

`enable_prefix_caching` can improve generation performance when the initial prompt content is repeated. That is the case with chat applications which resend the history of the conversations. Thanks to prefix caching, there is no need to reevaluate the same sequence of tokens. Thanks to that, first token will be generated much quicker and the overall utilization of resource will be lower. Old cache will be cleared automatically when entire free cache is consumed, so it is recommended to increase `cache_size` to get better performance advantage and store longer history of previous requests.

Another cache related option is `cache_eviction_config` which can help with latency of the long generation, but at the cost of accuracy. It's type is defined as follows:

```
message KVCrushConfig {
  enum AnchorPointMode {
    RANDOM = 0;
    ZEROS = 1;
    ONES = 2;
    MEAN = 3;
    ALTERNATING = 4;
  }
  optional uint64 budget = 2 [default = 0];
  optional AnchorPointMode anchor_point_mode = 3 [default = RANDOM];
  optional uint64 rng_seed = 4 [default = 0];
}
```

```
message CacheEvictionConfig {
  enum AggregationMode {
    SUM = 0; // In this mode the importance scores of each token will be
summed after each step of generation
    NORM_SUM = 1; // Same as SUM, but the importance scores are
additionally divided by the lifetime (in tokens generated) of a given token
in cache
  }

  optional AggregationMode aggregation_mode = 1 [default = NORM_SUM];
  required uint64 start_size = 2;
  required uint64 recent_size = 3;
  required uint64 max_cache_size = 4;
```

[Skip to main content](#)

```
optional KVCrushConfig kv_crush_config = 7;
}
```

Learn more about the algorithm and above parameters from [GenAI docs](#). Example of a graph with cache eviction configured:

```
input_stream: "HTTP_REQUEST_PAYLOAD:input"
output_stream: "HTTP_RESPONSE_PAYLOAD:output"

node: {
  name: "LLMExecutor"
  calculator: "HttpLLMCalculator"
  input_stream: "LOOPBACK:loopback"
  input_stream: "HTTP_REQUEST_PAYLOAD:input"
  input_side_packet: "LLM_NODE_RESOURCES:llm"
  output_stream: "LOOPBACK:loopback"
  output_stream: "HTTP_RESPONSE_PAYLOAD:output"
  input_stream_info: {
    tag_index: 'LOOPBACK:0',
    back_edge: true
  }
}
node_options: {
  [type.googleapis.com / mediapipe.LLMCalculatorOptions]: {
    models_path: "./",
    cache_eviction_config: {
      start_size: 32,
      recent_size: 128,
      max_cache_size: 672,
      kv_crush_config: {
        anchor_point_mode: RANDOM
        budget: 8
        rng_seed: 42
      }
    }
  }
}
input_stream_handler {
  input_stream_handler: "SyncSetInputStreamHandler",
  options {
    [mediapipe.SyncSetInputStreamHandlerOptions.ext] {
      sync_set {
        tag_index: "LOOPBACK:0"
      }
    }
  }
}
```

Scheduling settings

[Skip to main content](#)

In different use cases and load specification, requests and tokens scheduling might play a role when it comes to performance.

- `dynamic_split_fuse` [algorithm](#) is enabled by default to boost the throughput by splitting the tokens to even chunks. In some conditions like with very low concurrency or with very short prompts, it might be beneficial to disable this algorithm.
- Since `max_num_batched_tokens` defines how many tokens can a pipeline process in one step, when `dynamic_split_fuse` is disabled, `max_num_batched_tokens` should be set to match the model max context length since the prompt is not split and must get processed fully in one step.
- Setting `max_num_seqs` might also be useful in providing certain level of generation speed of requests already in the pipeline. This value should not be higher than `max_num_batched_tokens`.
- Scheduler configuration also accept sparse attention config with following options in `graph.pbtxt`:

```
enum SparseAttentionMode {
    TRISHAPE = 0;
    XATTENTION = 1;
}
optional SparseAttentionMode mode = 1 [default = TRISHAPE];
optional uint64 num_last_dense_tokens_in_prefill = 2;
optional uint64 num_retained_start_tokens_in_cache = 3;
optional uint64 num_retained_recent_tokens_in_cache = 4;
optional float xattention_threshold = 5;
optional uint64 xattention_block_size = 6;
optional uint64 xattention_stride = 7;
```

Description of parameters in that config can be found in GenAI docs about [SparseAttentionConfig](#).

Example of graph with sparse attention configured:

```
input_stream: "HTTP_REQUEST_PAYLOAD:input"
output_stream: "HTTP_RESPONSE_PAYLOAD:output"

node: {
  name: "LLMExecutor"
  calculator: "HttpLLMCalculator"
  input_stream: "LOOPBACK:loopback"
  input_stream: "HTTP_REQUEST_PAYLOAD:input"
  input_side_packet: "LLM_NODE_RESOURCES:llm"
  output_stream: "LOOPBACK:loopback"
  output_stream: "HTTP_RESPONSE_PAYLOAD:output"
  input_stream_info: {
```

[Skip to main content](#)

```

    }
    node_options: {
      [type.googleapis.com / mediapipe.LLMCalculatorOptions]: {
        models_path: "./",
        sparse_attention_config: {
          mode: TRISHAPE
          num_last_dense_tokens_in_prefill: 100
          num_retained_start_tokens_in_cache: 128
          num_retained_recent_tokens_in_cache: 1920
          xattention_threshold: 0.8
          xattention_block_size: 64
          xattention_stride: 8
        }
      }
    }
  }
  input_stream_handler {
    input_stream_handler: "SyncSetInputStreamHandler",
    options {
      [mediapipe.SyncSetInputStreamHandlerOptions.ext] {
        sync_set {
          tag_index: "LOOPBACK:0"
        }
      }
    }
  }
}

```

Note that the following options are ignored in Stateful servables (so in deployments on NPU): `cache_size`, `dynamic_split_fuse`, `max_num_batched_tokens`, `max_num_seq`, `enable_prefix_caching`, `cache_eviction_config`, `sparse_attention_config`

Output parsing settings

When using models with more complex templates and support for `tools` or `reasoning`, you need to pass `tool_parser` or `reasoning_parser` option that defines which parser should be used for processing model output and creating final response. Currently, model server supports following parsers:

Tool parsers:

- `hermes3` (also works for Qwen3 models)
- `llama3`
- `phi4`
- `mistral`
- `devstral`
- `gptoss`

[Skip to main content](#)

Reasoning parsers:

- `qwen3`

Note that using `tools` might require a chat template other than the original. We recommend using templates from the [vLLM repository](#) for `hermes3`, `llama3`, `phi4`, `mistral`, `devstral`, `gptoss`, and `qwen3coder` models (if available). Save the selected template as `chat_template.jinja` in the model directory and it will be used instead of the default one. If a template is not available for your model, please refer to the model's documentation or use the default template provided by the model server.

When `tool_parser` is used, it's possible to leverage tool guided generation with `enable_tool_guided_generation` option. That setting pushes the model to generate tool calls that matches the schemas specified in the `tools`.

Note: When `enable_tool_guided_generation` is set, but model server fails to load any tool schema from the request, the request will still be processed, but tool guided generation will be disabled.

OpenVINO runtime settings

`plugin_config` accepts a json dictionary of tuning parameters for the OpenVINO plugin. It can tune the behavior of the inference runtime. For example you can include there kv cache compression or the group size `{"KV_CACHE_PRECISION": "u8", "DYNAMIC_QUANTIZATION_GROUP_SIZE": "32"}`. It also holds additional options that are described below.

The LLM calculator config can also restrict the range of sampling parameters in the client requests. If needed change the default values for `best_of_limit` or set `max_tokens_limit`. It is meant to avoid the result of memory overconsumption by invalid requests.

Additional settings in plugin_config

As mentioned above, in LLM pipelines, `plugin_config` map holds not only OpenVINO device plugin options, but also additional pipeline configuration. Those additional options are:

- `prompt_lookup` - if set to `true`, pipeline will use [prompt lookup decoding](#) technique for sampling new tokens. Example: `plugin_config: '{"prompt_lookup": true}'`. With prompt lookup enabled it is recommended to disable `dynamic_split_fuse` and set `max num batched tokens` to be higher than model context length.

[Skip to main content](#)

- **MAX_PROMPT_LEN** (important for NPU users) - NPU plugin sets a limitation on prompt (1024 tokens by default), this options allows modifying this value. Example:

```
plugin_config: '{"DEVICE_PROPERTIES": { "NPU": {"MAX_PROMPT_LEN": 2048}}}'
```

Canceling the generation

In order to optimize the usage of compute resources, it is important to stop the text generation when it becomes irrelevant for the client or when the client gets disconnected for any reason. Such capability is implemented via a tight integration between the LLM calculator and the model server frontend. The calculator gets notified about the client session disconnection. When the client application stops or deliberately breaks the session, the generation cycle gets broken and all resources are released. Below is an easy example how the client can initialize stopping the generation:

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v3", api_key="unused")
stream = client.completions.create(model="model", prompt="Say this is a
test", stream=True)
for chunk in stream:
    if chunk.choices[0].text is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
    if some_condition:
        stream.close()
        break
```

Models Directory

In node configuration we set **models_path** indicating location of the directory with files loaded by LLM engine. It loads following files:

```
├─ openvino_detokenizer.bin
├─ openvino_detokenizer.xml
├─ openvino_model.bin
├─ openvino_model.xml
├─ openvino_tokenizer.bin
├─ openvino_tokenizer.xml
├─ tokenizer_config.json
└─ chat_template.jinja
```

Main model as well as tokenizer and detokenizer are loaded from **.xml** and **.bin** files and all of them are required. **tokenizer_config.json** and **chat_template.jinja** are loaded to read information required for chat template processing. Model directory may also contain

[Skip to main content](#)

exists, model server will use it to load default generation configuration for processing request to that model.

Additionally, Visual Language Models have encoder and decoder models for text and vision and potentially other auxiliary models.

This model directory can be created based on the models from Hugging Face Hub or from the PyTorch model stored on the local filesystem. Exporting the models to Intermediate Representation format is one time operation and can speed up the loading time and reduce the storage volume, if it's combined with quantization and compression.

We recommend using [export script](#) to prepare models directory structure for serving.

Check [tested models](#).

Input preprocessing

Completions

When sending a request to `/completions` endpoint, model server adds `bos_token_id` during tokenization, so **there is no need to add `bos_token` to the prompt**.

Chat Completions

When sending a request to `/chat/completions` endpoint, model server will try to apply chat template to request `messages` contents.

Loading chat template proceeds as follows:

1. If `chat_template.jinja` is present, try to load template from it.
2. If there is no `chat_template.jinja` and `tokenizer_config.json` exists, try to read template from its `chat_template` field. If it's not present, use default template.
3. If `tokenizer_config.json` exists try to read `eos_token` and `bos_token` fields. If they are not present, both values are set to empty string.

If both `chat_template.jinja` file and `chat_template` field from `tokenizer_config.json` are successfully loaded, `chat_template.jinja` takes precedence over `tokenizer_config.json`.

Template is not applied for calls to `/completions`, so it doesn't have to exist, if you plan to work only with `/completions`.

[Skip to main content](#)

Errors during configuration files processing (access issue, corrupted file, incorrect content) result in servable loading failure.

When working with tools, `/chat/completions` API accepts `tool_choice` parameter which gives additional control over model behavior related to tool calling. `tool_choice` accepts three special string values:

- `auto` - model is free to decide whether to call a tool or not
- `none` - model server will try to push the model not to call any tool
- `required` - model server will try to push the model to call at least one tool

Additionally `tool_choice` can be an object describing specific tool to be called. For more see [API reference](#).

Output processing

Support for more diverse response structure requires processing model output for the purpose of extracting specific parts of the output and placing them in specific fields in the final response.

When using `tools`, we need to distill `tool_calls` from model output and for reasoning - `reasoning_content`. In order to receive such response, you need to specify `tool_parser` or `reasoning_parser` as stated in [output parsing settings](#).

Limitations

There are several known limitations which are expected to be addressed in the coming releases:

- Metrics related to text generation are not exposed via `metrics` endpoint. Key metrics from LLM calculators are included in the server logs with information about active requests, scheduled for text generation and KV Cache usage. It is possible to track in the metrics the number of active generation requests using metric called `ovms_current_graphs`. Also tracking statistics for request and responses is possible. [Learn more](#)
- `logprobs` parameter is not supported currently in streaming mode. It includes only a single logprob and do not include values for input tokens
- using `tools` is not supported in configuration without Python

Some servable types introduce additional limitations:

Stateful servable limitations

- `finish_reason` not supported (always set to `stop`)

[Skip to main content](#)

- sequential request processing (only one request is handled at a time),
- only a single response can be returned. Parameter `n` is not supported.
- prompt lookup decoding is not supported
- `usage` is not supported in streaming mode
- **[NPU only]** beam_search algorithm is not supported with NPU. Greedy search and multinomial algorithms are supported.
- **[NPU only]** models must be exported with INT4 precision and `--sym --ratio 1.0 --group-size -1` params. This is enforced in the `export_model.py` script when the `target_device` in NPU.

Visual Language servable limitations

- works only on `/chat/completions` endpoint,
- does not work with `tools`,
- **[NPU only]** requests MUST include one and only one image in the messages context. Other request will be rejected.

©2025 Intel Corporation	OpenVINO OpenVINO™ Telemetry Glossary Case Studies	Legal Terms of Use Responsible AI	Privacy Cookies Privacy
-------------------------	--	---	---

Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.